Rapport de projet OpsCI

NACO - Catalogue de films

Application web orientée données et API

Élément	Détail
Étudiants	FERROUKH Mohamed Nassim - 21308499 ZEQO Ermal - 21315866
Technologies principales	React + Vite, FastAPI, TMDB, PostgreSQL
Déploiement	Docker Compose, Kubernetes avec Minikube, GitLab CI/CD
Année	2025 - 2026

Projet réalisé dans le cadre du module OpsCI.

Sommaire

1. Introduction
2. Cahier des charges et objectifs
3. Architecture générale
4. Choix des technologies
5. Front-end React + Vite
6. Back-end FastAPI et API TMDB
7. Base de données PostgreSQL et favoris
8. Docker et Docker Compose
9. Déploiement Kubernetes
10. GitLab CI/CD
11. Tests et validation
12. Difficultés rencontrées
13. Limites et améliorations possibles
14. Conclusion

1. Introduction

Ce projet avait pour but de développer une application web complète autour d'un catalogue de films. L'objectif n'était pas seulement d'afficher une page web, mais de construire une petite architecture proche d'un cas réel : un front-end, un back-end, une base de données, une API externe, puis une partie de déploiement avec Docker, Kubernetes et CI/CD.

Notre application s'appelle NACO. Elle permet d'afficher des films populaires, de rechercher des films, de charger plusieurs pages de résultats et d'ajouter certains films en favoris. Les données principales viennent de l'API TMDB, tandis que les favoris sont sauvegardés dans PostgreSQL.

Le projet a évolué progressivement. Au départ, le front-end était un front-end classique en HTML, CSS et JavaScript. Nous l'avons ensuite migré vers React avec Vite pour avoir une structure plus propre et plus facile à faire évoluer. Le back-end est fait avec FastAPI, et l'ensemble du projet peut être lancé en local avec Docker Compose ou déployé sur Kubernetes.

2. Cahier des charges et objectifs

Le sujet demandait de concevoir une application web orientée données et API. Les points obligatoires étaient notamment : catalogue de films dynamique, barre de recherche, architecture front-end / back-end / base de données, API HTTP, conteneurisation Docker, déploiement Kubernetes, CI/CD, README, rapport et vidéo de démonstration.

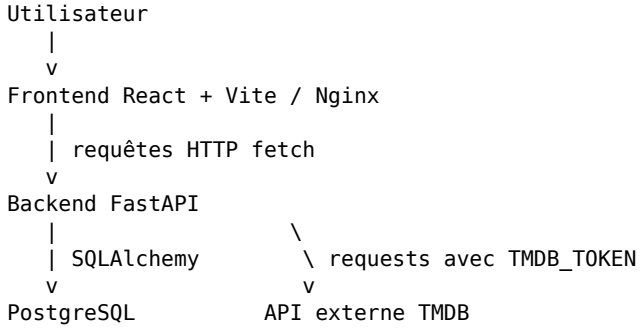
Les objectifs principaux retenus pour notre projet étaient donc :

- afficher un catalogue de films dynamique ;
- permettre une recherche de films depuis une barre de recherche ;
- avoir une communication fonctionnelle entre le front-end et le back-end ;
- utiliser une base de données obligatoire ;
- gérer au minimum les erreurs ;
- conteneuriser le front-end, le back-end et la base ;
- fournir des manifests Kubernetes avec Services, Ingress, ConfigMap et Secret ;
- mettre en place un pipeline GitLab CI/CD.

Nous avons aussi ajouté des fonctionnalités valorisées par le sujet : les favoris, la pagination, un cache côté back-end et une gestion plus complète des erreurs de l'API TMDB.

3. Architecture générale

L'architecture finale est séparée en trois parties principales : le front-end React, le back-end FastAPI et la base PostgreSQL. L'API externe TMDB est appelée uniquement par le back-end. Ce choix est important car le token TMDB ne doit pas être visible dans le code front-end ou dans le navigateur.



Dans cette organisation, le front-end ne connaît pas directement TMDB. Il appelle seulement notre API interne. Le back-end joue donc le rôle d'intermédiaire : il interroge TMDB, normalise les données et renvoie une réponse simple au front-end.

Composant	Rôle dans le projet
Frontend React/Vite	Interface utilisateur, affichage des films, recherche, pagination, favoris.
Backend FastAPI	API HTTP du projet, appels à TMDB, normalisation des données, gestion des erreurs.
PostgreSQL	Stockage persistant des films ajoutés aux favoris.
TMDB	Source externe pour les données de films.
Docker Compose	Lancement local de tous les services.
Kubernetes	Déploiement en cluster avec Services, Ingress, Secret et PVC.
GitLab CI/CD	Vérification automatique du projet à chaque push.

4. Choix des technologies

React + Vite

Nous avons choisi React avec Vite pour moderniser le front-end. React permet de gérer plus facilement l'état de l'application, par exemple la liste des films, les favoris, la recherche, les messages d'erreur et le chargement. Vite est pratique car il démarre rapidement en développement et génère un build de production simple à servir avec Nginx.

FastAPI

FastAPI a été choisi pour le back-end car il est simple à mettre en place, rapide et bien adapté à une API HTTP. Il permet de définir clairement les routes, les paramètres de requête et les réponses JSON. Il s'intègre aussi facilement avec SQLAlchemy pour communiquer avec PostgreSQL.

TMDB

Pour la source de données, nous avons choisi l'API externe TMDB. Ce choix nous a permis de travailler avec de vraies données de films : titres, descriptions, dates de sortie, images et identifiants. L'utilisation de TMDB oblige aussi à gérer un token d'API, ce qui correspond bien à un cas professionnel réel.

PostgreSQL

PostgreSQL a été utilisé pour respecter l'obligation d'avoir une base de données. Dans notre application, il sert à sauvegarder les favoris. Nous avons choisi une base SQL car les données à stocker sont simples et structurées : un id, un tmdb_id, un titre, une année, une description et une image.

5. Front-end React + Vite

Le front-end a été migré d'une version classique HTML/CSS/JavaScript vers React. Le fichier `index.html` sert maintenant de point d'entrée Vite, `src/main.jsx` initialise React, et `src/App.jsx` contient la logique principale de l'application.

Dans `App.jsx`, une variable `API_BASE` permet de changer l'URL de l'API selon l'environnement. En local avec Docker Compose, le front-end appelle `http://127.0.0.1:8000`. En Kubernetes, il appelle `/api`, puis l'Ingress redirige vers le back-end.

```
const API_BASE =
  import.meta.env.VITE_API_BASE_URL || "http://127.0.0.1:8000";
```

Les appels au back-end sont ensuite faits avec `fetch`. Par exemple, la page des films appelle `/movies` avec un paramètre page, la recherche appelle `/movies/search`, et les favoris utilisent les routes `/favorites`.

Les fonctionnalités côté front-end sont :

- affichage dynamique des films ;
- barre de recherche connectée au back-end ;

- bouton Charger plus pour utiliser la pagination ;
- affichage des états loading et erreur ;
- ajout et suppression de films favoris ;
- page séparée pour consulter les favoris.

Le style de l'application a été conservé et adapté dans `App.css`. L'interface reprend une présentation inspirée des catalogues de streaming, avec une barre de navigation, un hero banner, des cartes de films et une page favoris.

6. Back-end FastAPI et API TMDB

Le back-end expose une API HTTP utilisée par React. Il est aussi responsable des appels vers TMDB. Le token TMDB est chargé depuis les variables d'environnement et reste côté serveur.

Route	Méthode	Rôle
`/`	GET	Vérifier que le back-end fonctionne.
`/movies`	GET	Récupérer les films populaires depuis TMDB avec pagination.
`/movies/search`	GET	Rechercher des films avec une requête q et une page.
`/favorites`	GET	Lister les films favoris stockés dans PostgreSQL.
`/favorites`	POST	Ajouter un film aux favoris.
`/favorites/{tmdb_id}`	DELETE	Supprimer un favori à partir de son identifiant TMDB.
`/export/movies.json`	GET	Exporter plusieurs pages de films populaires en JSON.

Une fonction `normalize_movie` transforme les réponses TMDB en objets plus simples pour le front-end. Cela évite que React dépende directement du format exact de TMDB.

La gestion des erreurs a été travaillée dans la fonction `tmdb_get` : token manquant ou invalide, limite de requêtes, timeout, erreurs réseau et erreurs serveur. En cas de problème, le back-end renvoie une erreur HTTP claire au front-end.

Les cas traités sont notamment :

- `401` si le token TMDB est absent ou invalide ;
- `429` si TMDB renvoie une limite de requêtes ;
- `504` en cas de timeout ;
- `502` en cas d'erreur réseau ;
- retries avec backoff pour certains cas temporaires.

Un cache simple en mémoire a aussi été ajouté pour les films populaires et les recherches. Cela évite de refaire exactement les mêmes appels à TMDB pendant un court intervalle.

7. Base de données PostgreSQL et favoris

PostgreSQL est utilisé pour stocker les favoris. La connexion se fait avec SQLAlchemy à partir de la variable `DATABASE_URL`. Au démarrage, FastAPI crée automatiquement la table si elle n'existe pas encore.

Champ	Description
`id`	Identifiant interne du favori.
`tmdb_id`	Identifiant du film dans TMDB. Il est unique pour éviter les doublons.
`title`	Titre du film.
`year`	Année de sortie.
`description`	Résumé du film.
`image_url`	URL de l'affiche du film.

L'ajout d'un favori vérifie d'abord si le film existe déjà dans la base. Si c'est le cas, le back-end ne crée pas de doublon et renvoie simplement le favori existant. La suppression se fait à partir du `tmdb_id`, ce qui est pratique côté front-end.

En Docker Compose, la persistance est assurée par le volume `naco_pgdata`. En Kubernetes, elle est assurée par un `PersistentVolumeClaim`.

8. Docker et Docker Compose

La conteneurisation permet de lancer l'application sans installer manuellement toutes les dépendances sur la machine. Nous avons créé un Dockerfile pour le back-end, un Dockerfile pour le front-end, puis un `docker-compose.yml` pour lancer les trois services ensemble.

Service	Image / Build	Port	Rôle
backend	build `./backend`	`8000:8000`	API FastAPI lancée avec Uvicorn.
frontend	build `./frontend`	`5173:80`	Build React servi par Nginx.
db	`postgres:16`	`5432:5432`	Base PostgreSQL.

Le back-end lit ses variables depuis `backend/.env`. Le front-end est construit par Node puis servi par Nginx. PostgreSQL utilise un volume pour garder les données entre deux redémarrages.

```
docker compose up --build
```

En local Docker Compose, il faut utiliser cette configuration dans `frontend/.env.production` :

```
VITE_API_BASE_URL=http://127.0.0.1:8000
```

En Kubernetes, cette valeur devient `/api` car le front-end et le back-end passent par l'Ingress.

9. Déploiement Kubernetes

La partie Kubernetes a été testée avec Minikube. Le but était de déployer la même architecture que Docker Compose, mais sous forme de ressources Kubernetes.

Les ressources principales sont :

- Namespace `naco` pour isoler les ressources ;
- ConfigMap pour la configuration non sensible ;
- Secret pour `TMDB_TOKEN`, `POSTGRES_PASSWORD` et `DATABASE_URL` ;
- Deployment back-end avec 2 replicas ;
- Deployment front-end avec 2 replicas ;
- Deployment PostgreSQL avec 1 replica ;
- Services `ClusterIP` pour relier les composants ;
- PVC pour persister les données PostgreSQL ;
- Ingress pour exposer l'application via `naco.local` ;
- HPA pour montrer une stratégie de scaling.

Commandes principales utilisées :

```
minikube start
minikube addons enable ingress

eval $(minikube docker-env)
docker build -t naco-backend:latest ./backend
docker build -t naco-frontend:latest ./frontend

kubectl apply -f k8s/
```

Nous avons séparé l'Ingress en deux ressources. Une première route `/api/...` vers le back-end avec rewrite, et une deuxième route `/` vers le front-end. Au début, un seul Ingress avec une règle de rewrite

cassait les fichiers statiques React dans `/assets`. La séparation des deux Ingress a corrigé ce problème.

```
http://naco.local                -> frontend
http://naco.local/api/movies     -> backend /movies
http://naco.local/api/favorites  -> backend /favorites
```

Le scaling est possible sur le front-end et le back-end car ils ne stockent pas directement les données localement. PostgreSQL reste à un seul replica dans notre projet, car c'est un composant stateful. Pour une vraie production, on utiliserait plutôt un StatefulSet ou une base managée.

10. GitLab CI/CD

Le projet a été poussé sur GitLab. Un fichier `.gitlab-ci.yml` a été ajouté pour automatiser plusieurs contrôles à chaque push.

Stage	Job	Ce qui est vérifié
test	`backend_check`	Installation des dépendances Python et vérification de `main.py` avec `py_compile`.
build	`frontend_build`	Installation npm et build React/Vite avec `VITE_API_BASE_URL=/api`.
docker	`docker_backend`	Vérification que le Dockerfile back-end est présent et cohérent.
docker	`docker_frontend`	Vérification que le Dockerfile front-end est présent et cohérent.
deploy	`deploy_kubernetes`	Étape manuelle prévue pour appliquer les manifests Kubernetes.

Au départ, nous avons essayé de construire et pousser les images Docker directement depuis GitLab CI. Cependant, le runner disponible ne permettait pas d'utiliser Docker-in-Docker correctement. L'erreur indiquait que le job ne pouvait pas se connecter au daemon Docker.

Nous avons donc adapté le pipeline : les images Docker ont été construites et validées localement, et la CI vérifie les Dockerfiles, le back-end et le build front-end.

L'étape `deploy_kubernetes` reste manuelle. C'est volontaire : le cluster utilisé est Minikube, donc il tourne localement sur notre machine. GitLab ne peut pas y accéder sans configuration supplémentaire comme un `KUBECONFIG` sécurisé ou un GitLab Kubernetes Agent.

11. Tests et validation

Plusieurs tests manuels et techniques ont été faits pour vérifier le projet :

- test de `GET /movies` pour vérifier les films populaires ;
- test de `GET /movies/search` avec une requête de recherche ;
- test de `GET /favorites` sur une base vide ;
- test d'ajout d'un favori depuis React ;
- test de suppression d'un favori ;
- test de persistance après redémarrage Docker Compose ;
- test de `docker compose up --build` ;
- test du déploiement Kubernetes sur Minikube ;
- test de `http://naco.local` et des routes `/api` ;
- test du pipeline GitLab CI/CD.

Un exemple de réponse attendue pour une base de favoris vide est :

```
{
  "status": "ok",
  "count": 0,
  "results": []
}
```

12. Difficultés rencontrées

Problème	Solution apportée
Gestion du token TMDB	Le token a été gardé côté back-end dans `.env` et dans un Secret Kubernetes, jamais dans le code React.
URL API différente selon l'environnement	Utilisation de `VITE_API_BASE_URL` : URL locale pour Docker Compose, `/api` pour Kubernetes.
Ingress Kubernetes qui affichait une page blanche	Séparation en deux Ingress : un pour `/api` avec rewrite, un pour `/` sans rewrite.
Docker-in-Docker indisponible dans GitLab	Adaptation du pipeline pour vérifier les Dockerfiles et garder le build image validé localement.
Persistance PostgreSQL	Utilisation d'un volume Docker puis d'un PVC Kubernetes.

Ces difficultés ont été utiles car elles correspondent à des problèmes assez réalistes : configuration par environnement, secrets, routage Ingress et limites d'un runner CI partagé.

13. Limites et améliorations possibles

L'application fonctionne, mais plusieurs améliorations seraient possibles :

- ajouter de vrais tests automatisés avec pytest pour le back-end ;
- ajouter des tests front-end avec Vitest ou Playwright ;
- ajouter Redis pour un cache plus robuste que le cache mémoire ;
- ajouter une authentification utilisateur pour avoir des favoris par compte ;
- mettre en place un vrai registre d'images Docker ;
- automatiser le déploiement Kubernetes avec un `KUBECONFIG` ou GitLab Agent ;
- utiliser un StatefulSet ou une base PostgreSQL managée en production ;
- ajouter une page détail film avec plus d'informations TMDB ;
- améliorer le README final pour faciliter le lancement par une personne externe.

Une remarque importante concerne les secrets : les vrais tokens ne doivent jamais être commités. Si un vrai token a été placé par erreur dans un fichier d'exemple pendant les tests, il faut le remplacer par une valeur fictive et régénérer le token côté TMDB.

14. Conclusion

Ce projet nous a permis de construire une application web complète en partant d'une interface simple et en allant jusqu'à une architecture de déploiement. Le résultat final contient un front-end React/Vite, un back-end FastAPI, une API externe TMDB, une base PostgreSQL, Docker Compose, Kubernetes et GitLab CI/CD.

Le projet répond aux attentes principales du sujet : catalogue dynamique, recherche, communication front/back, base de données, Docker, Kubernetes, CI/CD et documentation. Les choix techniques restent simples, mais ils sont cohérents avec l'objectif : montrer une application réaliste, compréhensible et déployable.

La partie Kubernetes et la CI/CD ont aussi montré que le déploiement ne se limite pas à écrire du code. Il faut gérer les variables d'environnement, les secrets, les différences entre local et cluster, et les contraintes des outils utilisés. C'est justement ce qui rend le projet proche d'une situation professionnelle.

15. Références

- Sujet du projet OpsCI 2025 : *Conception et déploiement d'une application Web orientée données et API*.
- Documentation FastAPI, React, Vite, Docker, Kubernetes et GitLab CI/CD consultée pendant le développement.
- Documentation TMDB pour les endpoints de films populaires et de recherche.